

TrackNet CITYVISION Frontend Architecture and Implementation Process

Interview brief for a Flutter Web ANPR dashboard

This document explains what the frontend does, how data moves through it, which functions and methods are used, and how the demo layer can be replaced by a future backend without rebuilding the interface.

Interview topic	Answer in one sentence
What is the product?	A responsive TrackNet / CITYVISION urban traffic intelligence dashboard for camera monitoring, ANPR detections, vehicle journeys, traffic analytics, alerts and network operations.
What is the core design choice?	Widgets depend on typed domain models and Riverpod providers, while demo or backend repositories supply the data behind the same visual frontend.
What is the practical benefit?	Demo behavior is previewable now, and real REST, WebSocket or streaming data can be connected later by replacing the repository adapter.

Architecture at a glance

```
Flutter Web entry point
-> ProviderScope + TrackNetApp
-> MaterialApp.router + go_router
-> Riverpod providers select active DataMode
-> DemoDashboardRepository OR BackendDashboardRepository
-> typed models and RepositoryState
-> feature pages and shared widgets
-> responsive cards, tables, charts, maps and interaction panels
```

The visual language is a calm neutral light theme: white cards, zinc text, thin borders, olive-green accents, readable hierarchy and restrained status colors. The product identity and ANPR content remain TrackNet / CITYVISION.

How to explain the project in 30 seconds

I built a Flutter Web dashboard with a feature-first structure. Riverpod owns reactive state and injects repositories, go_router owns deep links and browser history, and typed models keep the widgets independent from temporary demo arrays. Demo mode runs seeded simulations inside the mock repository. Backend mode starts with explicit loading, empty, error or offline states and is ready for a transport adapter when the real model and API contract are available.

1. Complete frontend process

The application follows a predictable path from the browser entry point to the rendered widget and back to the repository when a user interacts with the screen.

Step	What happens	Main Dart concepts
1. Start	main.dart calls runApp with ProviderScope so every feature can read shared providers.	runApp, ProviderScope
2. Configure	The app reads DATA_MODE from dart-define and chooses demo or backend behavior without changing UI code.	String.fromEnvironment, enum
3. Route	MaterialApp.router resolves dashboard, cameras, vehicle, analytics, alerts, network and settings paths.	GoRouter, context.go
4. Load	A repository provider exposes RepositoryState: loading, ready, empty, error or offline.	AsyncValue, StateNotifier
5. Select	Widgets watch only the model slice they need, so a detection tick does not rebuild every page.	ref.watch, select
6. Render	Feature widgets build cards, tables, charts, map layers and responsive navigation from typed models.	build, LayoutBuilder, CustomPainter
7. Interact	Search, filter, resolve, replay, reconnect and layer actions call provider methods or repository methods.	ref.read, Future, Stream
8. Dispose	Timers, stream controllers, animation controllers and subscriptions are closed with the widget or repository.	dispose, close, cancel

Example interaction flow: resolve an alert

1. The user taps Resolve in the alert detail panel.
2. The widget calls `ref.read(alertControllerProvider.notifier).resolve(alertId)`.
3. The controller calls `AlertRepository.resolveAlert`, which updates the active data source.
4. Riverpod publishes the changed alert list and counts.
5. Only the alert list, counts, notification panel and related map marker rebuild.
6. A toast confirms the action. Backend mode can later map the same method to a real API or stream command.

Simulation and update timing

Process	Demo behavior	Why it is isolated
Live detection tick	Approximately every 2.6 seconds; adds a seeded or generated detection and refreshes feed/map data.	Temporary generators remain in MockDashboardRepository.
Clock and uptime	Every second; updates the live header clock and platform uptime.	UI receives a small time model instead of owning a timer.
KPI and analytics	Approximately every 5.2 seconds; refreshes totals, chart series and traffic summaries.	Charts consume AnalyticsSnapshot.
Heat and zones	Approximately every 8 seconds; refreshes heat intensity and zone traffic values.	Map layers consume MapLayerState and Zone models.
Boot sequence	About 2 to 3 seconds with a safe Flutter fallback animation.	Boot state is presentation state, not backend data.

2. Architecture and data design

Feature-first project structure

```
lib/
  app/ theme/ routing/ models/ providers/ repositories/ data_sources/
  shared_widgets/ animations/ map/ charts/ demo/ demo_data/ mock_repositories/
  features/dashboard/ cameras/ vehicle/ analytics/ alerts/ network/ settings/
```

Typed domain models

Model	What it represents	Used by
Camera	ID, name, location, status, speed, accuracy, latest plate and latest detection.	Dashboard, cameras, network, map
Detection	Plate result, confidence, timestamp, camera, vehicle type and safety status.	Live feed, vehicle log, alerts
Vehicle	Plate identity, type, color, first and last seen context.	Vehicle intelligence
Journey	Captured route, distance, duration, average speed and timeline events.	Vehicle route and replay
Alert	Severity, reason, location, time, status and resolution metadata.	Alert center, notifications, map
Zone	Traffic area, count, speed, density and map geometry.	Analytics and heat map
AnalyticsSnapshot	Traffic volume, average speed, camera density, vehicle split and OD flow.	Charts and analytics
PlatformStatus	Uptime, cameras tracked, tile status, routes indexed and alerts raised.	Settings and header
SettingsState	Simulation, particles, alerts, feed engine, grid and reconnect preferences.	Settings
MapLayerState	Camera, vehicle, heat, zones, routes and particle visibility.	Map controls

Repository boundaries

Interface	Responsibility
DashboardRepository	Aggregate dashboard snapshot, live stream state, simulation controls and shared lifecycle.
CameraRepository	Camera list, camera details, feed state, filters, polling and reconnect.
VehicleRepository	Plate lookup, suggestions, vehicle details, journey and replay state.
DetectionRepository	Detection history, filters, search, sorting and selected event.
AlertRepository	Alert list, counts, detail, resolve and dismiss actions.
AnalyticsRepository	Chart series, hour lens, zones, flows and analytics refresh.
NetworkRepository	Camera registry, pagination, sorting, statuses and poll registry.
MapRepository	Map shapes, markers, layers, heat values and route geometry.
SettingsRepository	Settings state, reset demo data and reconnection monitor.

Two data modes

Mode	Source and behavior	Missing data behavior
DataMode.demo	MockDashboardRepository owns faithful seeded CITYVISION demo records, timers, procedural feeds, particles, alerts and fake map values.	Never calls a backend; reset demo restores the seeded state.
DataMode.backend	BackendDashboardRepository owns state and delegates to a future BackendTransport interface.	Shows loading, empty, error, offline or disconnected states. It does not fall back to demo values.

The UI does not know whether a value came from a seeded fixture, REST response, WebSocket event or stream. It only reads typed models and repository state. This is the main migration seam for the future model and backend.

3. Functions and methods used

These are the Flutter and Dart methods that do the work in the frontend. The important pattern is that each method has one clear job and state changes flow through providers.

Function or method	Purpose in this app	Interview explanation
<code>build(BuildContext)</code>	Creates the widget tree for cards, pages, controls and panels.	Flutter calls build again when a watched value changes.
<code>ref.watch(provider)</code>	Subscribes a widget to reactive Riverpod state.	The widget rebuilds when that provider changes.
<code>ref.watch(provider.select(...))</code>	Subscribes to one field or slice of a large model.	This limits rebuilds and keeps updates smooth.
<code>ref.read(provider)</code>	Performs an action without subscribing to the result.	Used for button taps, commands and one-off repository calls.
<code>ref.listen(provider, callback)</code>	Runs side effects such as toast messages or navigation.	The view stays declarative while effects stay controlled.
Future / async / await	Handles plate lookup, details, reconnect and backend requests.	Loading and error states are explicit instead of blocking the UI.
StreamController / StreamProvider	Carries live detections, camera feed updates and backend events.	A stream is the replaceable seam for WebSocket or model output.
Timer.periodic	Runs demo-only clock, detection, analytics and heat schedules.	Timers live in the repository and are cancelled on dispose.
<code>copyWith(...)</code>	Creates a changed immutable model while preserving other fields.	State updates stay predictable and easy to test.
<code>context.go(...)</code> / GoRoute	Navigates between feature pages and deep links.	Browser back and forward remain available through go_router.
LayoutBuilder / MediaQuery	Chooses desktop, tablet or mobile layout and navigation.	Important content stays reachable at every width.
CustomPainter.paint	Draws the synthetic map, roads, landmarks, markers, heat and OD flows.	The map is pure Flutter and has no WebView or JavaScript dependency.
AnimationController / dispose	Drives feed motion, route replay and small UI transitions.	Controllers are stopped and disposed to avoid leaks.

State flow in plain language

```

user action -> provider/controller method -> repository method
              -> immutable model/state update
              -> Riverpod notification
              -> focused widget rebuild
              -> toast, route change or visual update
  
```

Why this avoids a full rebuild

A live detection tick updates the detection stream, feed state and selected map markers. KPI cards watch analytics fields, the clock watches platform time, and settings watches only its own state. Riverpod selectors keep unrelated cards and tables from rebuilding on every tick.

Interaction examples

User action	Method chain	Visible result
Search a plate	Text input -> debounce/search controller -> VehicleRepository.findVehicle	Suggestions, status, vehicle card and journey timeline.
Replay journey	Replay button -> JourneyReplayController -> AnimationController	Route marker advances through captured timeline events.
Toggle a map layer	Switch -> MapLayerState.copyWith -> provider update	Camera, vehicle, heat, zone or route drawing changes immediately.
Reconnect camera	Button -> CameraRepository.reconnect -> ReconnectionState	Loading/reconnecting/success/error state and camera status update.

4. Feature-by-feature process

Feature	Frontend process	Preserved behavior
Dashboard	Reads KPI snapshot, live detections, top cameras and map state; controls pause/resume and map layers.	Five KPIs, live feed, top traffic cameras, synthetic city map and offline locator.
Live Cameras	Watches 12 camera models; rotates feeds, opens inspection sheet and navigates to camera or plate context.	Status, latest plate/detection, procedural feed visuals and reconnect workflow.
Vehicle Intelligence	Searches plate suggestions, loads vehicle and journey models, filters detection history and drives replay.	Distance, duration, average speed, camera count, route map, timeline and city log.
Traffic Analytics	Hour lens selects AnalyticsSnapshot; fl_chart renders series and CustomPainter renders OD flow and heat layers.	06:00, 09:00, Now, 12:00, 18:00, 21:00 controls and all four chart meanings.
Alert Center	Filters alerts by severity, opens detail/map context, resolves one/all and listens for notifications.	High, Medium, Low, counts, toast notifications and map location.
Camera Network	Filters and searches registry, sorts columns, pages through rows and polls the registry.	All/Online/Warning/Offline filters, page sizes 10/15/20/50/All and network map.
Settings	Toggles settings state, shows platform status, watches reconnection monitor and resets demo data.	Simulation, particles, automatic alerts, feed engine, ambient grid and status metrics.

Navigation and deep links

The shell keeps the left navigation on desktop and a compact drawer or bottom navigation on smaller screens. Routes are /dashboard, /cameras, /vehicle, /analytics, /alerts, /network and /settings. Query parameters preserve camera, plate, alert, zone and hour context, so a selected record can be shared and browser back/forward works.

Responsive rules

- Desktop uses the persistent sidebar, topbar, KPI row and multi-column card grid.
- Tablet reduces columns and keeps priority controls visible in a compact navigation treatment.
- Mobile stacks cards, uses horizontal scrolling for wide tables and keeps map controls reachable.
- SafeArea, LayoutBuilder, MediaQuery and scroll views prevent clipped content around browser and device insets.

Notification and profile behavior

The topbar notification control opens a compact panel with unread count, severity, timestamp, dismiss action and a View all action. The operator icon opens a small profile panel with A. Sharma, role and shift details. Both panels are presentation widgets driven by the same alert and operator state as the rest of the app.

5. Rendering, theme and ANPR safety

Visual implementation

Area	Implementation
Theme	A light Material theme with #F9F9F9 background, white surfaces, zinc text, olive-green primary accents, warm beige support tones and restrained warning colors.
Typography	Google Fonts provides a clean modern sans-serif with strong KPI numbers and compact labels.
Cards and tables	Rounded white cards, subtle shadows, thin neutral dividers, restrained badges and readable table headers.
Charts	fl_chart renders traffic volume, average speed, camera density and vehicle type split with the original semantic series.
Map	Pure Flutter InteractiveViewer plus CustomPainter for roads, landmarks, markers, zones, heat, particles, routes and OD flows.
Animation	Built-in Flutter transitions animate KPI changes, selection, feed motion and journey replay. Rive/Lottie slots have safe fallbacks.

Map scroll fix

The map uses explicit zoom controls and pan gestures while allowing page scroll to pass through when the user scrolls vertically over the map. The map is not allowed to hijack normal page scrolling or become an accidental maximize/minimize target. Sidebar collapse arrow controls were removed so navigation remains stable.

ANPR data safety

The domain model keeps camera, timestamp, confidence and context for every detection. The Indian plate format is validated with `^[A-Z]{2}[0-9]{1,2}[A-Z]{1,2}[0-9]{4}$`, while OCR normalization can flag O/0, I/1, B/8, S/5 and Z/2 confusion for review. The UI supports NO_PLATE_DETECTED, LOW_CONFIDENCE_UNREADABLE, NORMAL, FLAGGED, BLACKLISTED, TAMPER_REVIEW and CLONE_REVIEW. It never guesses an unreadable plate or automatically creates an enforcement action.

States users can see

State	Meaning in the interface
Loading	The repository has started a request or stream connection.
Empty	The backend is available but there are no records for the current query.
Error	A request or mapping operation failed and the UI offers a retry path.
Offline / disconnected	The data source is unavailable; no demo values are substituted in backend mode.
Reduced motion	Animation durations and particle/feed motion are reduced while core controls remain usable.

6. Backend integration process

Backend mode is intentionally an adapter boundary. The frontend defines what it needs without inventing a URL, authentication scheme or JSON contract before the model team provides those details.

Current integration seam

```
BackendDashboardRepository
-> BackendTransport (future REST/WebSocket/stream implementation)
-> DTO parsing and DTO-to-domain mapping
-> typed domain models
-> Riverpod repository providers
-> unchanged pages, charts, tables, map and routes
```

Future backend step	What to add	What stays unchanged
Define contract	Agree DTO fields, transport events, auth and error semantics with the backend/model team.	Widgets and feature routes.
Implement transport	Implement BackendTransport methods for fetch, watch, journey lookup, camera feed, reconnect, resolve and settings.	Repository interfaces.
Map DTOs	Convert DTOs into Camera, Detection, Vehicle, Journey, Alert and analytics domain models.	Typed model consumers.
Inject adapter	Provide BackendDashboardRepository when DATA_MODE=backend.	Theme, layout, charts, tables and map.
Verify states	Test loading, empty, error, offline, reconnect and partial stream behavior.	Existing UI states and controls.

How to switch modes

```
flutter run -d chrome --dart-define=DATA_MODE=demo
flutter run -d chrome --dart-define=DATA_MODE=backend
```

The same routes and widgets run in both modes.

Why this is backend-ready

- Demo constants, random generators, simulation timers and fake map generation stay in the demo layer.
- Widgets receive typed models through providers instead of reading arrays directly.
- Backend mode never silently fills unavailable fields with temporary demo values.
- DTO mapping is a deliberate boundary where future model output can be normalized and validated.
- Repository methods provide one stable vocabulary for the UI even if transport changes from REST to streaming.

Demo data location

Look in lib/demo, lib/demo_data and lib/mock_repositories for the seeded cameras, detections, vehicles, journeys, alerts, zones, map shapes, chart values and simulation behavior. Look in lib/repositories and lib/data_sources for the replaceable contracts and backend adapter boundary.

7. Validation and interview preparation

Validation performed

Check	Result
dart format	Completed on the Flutter source.
flutter analyze	Clean; no analyzer errors.
flutter test	All automated tests passed.
flutter build web	Demo and backend configurations build successfully.
Route smoke check	All seven routes open, with deep-link query parameters preserved.
Responsive behavior	Desktop, tablet and mobile layouts use the same data providers and keep controls reachable.
Backend safety	Backend mode exposes loading/empty/error/offline states and does not fall back to demo data.
Interaction coverage	Filters, searches, pagination, charts, map layers, replay, alerts, reconnect and settings are wired.

60-second interview answer

TrackNet CITYVISION is a Flutter Web ANPR dashboard built with a feature-first architecture. `MaterialApp.router` and `go_router` provide the shell and deep links, while Riverpod owns state, repository injection and focused rebuilds. The UI consumes typed camera, detection, vehicle, journey, alert, zone, analytics and platform models. In demo mode, a mock repository keeps all seeded data, simulation timers, procedural feeds and synthetic map behavior. In backend mode, a backend repository exposes explicit loading, empty, error and offline states and delegates future REST, WebSocket or stream details to a transport adapter. Charts use `fl_chart`, maps and OD flows use pure Flutter `CustomPainter`, and responsive widgets adapt through `LayoutBuilder`, `MediaQuery` and `SafeArea`. That separation lets the backend replace the data source without a visual rebuild of the product.

Likely follow-up questions

Question	Concise answer
Why Riverpod?	It makes dependencies explicit, supports provider overrides and lets widgets select only the state they need.
Why repositories?	They isolate transport and demo behavior from UI, so the same feature can consume seeded or live data.
How do you avoid jank?	Focused selectors, bounded streams, disposed timers/controllers and animation only for visible content.
Why CustomPainter for the map?	The source has a synthetic map, so pure Flutter preserves its behavior without <code>WebView</code> or <code>JavaScript</code> .
How do you handle unreadable plates?	Keep the detection with explicit status and confidence; never fabricate a plate number.
How will the backend connect?	Implement <code>BackendTransport</code> and DTO mapping, then inject <code>BackendDashboardRepository</code> under <code>DATA_MODE=backend</code> .
What does the UI do when data is missing?	It shows loading, empty, error, offline or disconnected states; it never silently uses demo values in backend mode.

Useful project locations

Location	Purpose
lib/main.dart	Application entry point and ProviderScope.
lib/app/	TrackNetApp, shell and MaterialApp.router.
lib/routing/	go_router route table and deep-link handling.
lib/models/	Typed domain models and state objects.
lib/providers/	Riverpod providers, controllers, filters and selectors.
lib/repositories/	Repository interfaces and backend adapter boundary.
lib/mock_repositories/ and lib/demo_data/	Demo fixtures, simulation and seeded behavior.
lib/features/	Dashboard, cameras, vehicle, analytics, alerts, network and settings UI.
README.md	Run commands, architecture overview, backend handoff and limitations.

Current limitation: the backend transport contract is intentionally unimplemented until the real model and backend API or stream schema are supplied. The frontend is ready for that adapter, but it does not invent network endpoints, authentication details or JSON fields.